Lycée Duhamel du Monceau. Niveau : Première.		
	Numérique et Sciences Informatiques	
Manipulation de fichiers. Gestion des exceptions.		

I) <u>INTRODUCTION</u> :

Jusqu'à présent, les programmes réalisés ne traitaient qu'un très petit nombre de données. Nous pouvions donc à chaque fois inclure ces données dans le corps du programme lui-même (par exemple dans une liste). Cette façon de procéder devient tout à fait inadéquate lorsque l'on souhaite traiter une quantité d'informations plus importante.

II) <u>PRINCIPE GENERAL</u> :

Il est possible de manipuler depuis Python des données se trouvant dans un fichier autre que celui dans lequel nous sommes en train de travailler.
Un fichier est considéré depuis Python comme une suite de caractères (ou plus précisément d'octets selon la version de Python). Ainsi, les manipulations des fichiers se feront grâce aux commandes qui permettent de manipuler des chaînes de caractères.
Un fichier existant peut être ouvert en lecture (mode 'r') ou de façon à y ajouter des données à la suite de celles qui s'y trouvent déjà (mode 'a'). Pour créer un fichier et y écrire des données, on l'ouvre en écriture (mode 'w').
Dans tous les cas, on commence par ouvrir le fichier, on manipule ses données, puis on le referme.

Les syntaxes sont les suivantes :

Ouverture d'un fichier en lecture	**fichier=open("...", 'r')**
Ouverture d'un fichier en écriture	**fichier=open("...", 'w')**
Ouverture d'un fichier existant pour y ajouter des données	**fichier=open("...", 'a')**
Fermeture	**fichier.close()**

• Le nom fichier est arbitraire, on peut le remplacer par (presque) n'importe quoi.
• Les points de suspension doivent en pratique être remplacés par le chemin d'accès complet du fichier, c'est-à-dire l'emplacement du fichier donné par une lettre de lecteur, des dossiers, ainsi qu'un nom et une extension de fichier, le caractère de séparation étant le slash /.
Par exemple, C:/Users/a_durand/Documents/essais_python/fichiers1/test.txt est un chemin d'accès complet.
Si, lors d'une session, nous ne travaillons qu'avec des fichiers se trouvant tous dans le même répertoire, on peut faire de ce répertoire le répertoire courant pour n'avoir ensuite qu'à préciser des chemins d'accès relatifs. Pour cela, on utilise la commande chdir (change directory) du module os.

Par exemple, pour que le répertoire courant soit C:/Users/a_durand/Documents/essais_python, on écrit :

```
>>> from os import chdir
>>> chdir("C:/Users/a_durand/Documents/essais_python")
```

Pour ouvrir en écriture le fichier de chemin d'accès complet :
C:/Users/a_durand/Documents/essais_python/fichiers1/test.txt
Il suffira alors d'écrire fichier=open("fichiers1/test.txt", 'w').

<u>Remarque</u> : éviter les caractères accentués et spéciaux dans les noms des fichiers manipulés.

Dans Mes documents, créer un dossier essais_python, puis deux sous-dossiers : fichiers1 (pour y placer les fichiers créés) et fichiers2 (pour y placer les fichiers déjà existants et que nous allons manipuler).

Dans tous les programmes suivants, il ne faudra pas oublier d'indiquer le répertoire courant au début du script :

```
from os import chdir
chdir("P:/Mes documents/essais_python")
```

III) ÉCRITURE DANS UN FICHIER :

Lors de l'ouverture d'un fichier (en écriture ici) par fichier=open("fichiers1/test.txt", 'w'), nous créons un objet-fichier nommé ici fichier, qui permet de faire référence au fichier informatique. Cet objet-fichier se trouve dans la mémoire vive de votre ordinateur (le contenu s'efface une fois l'ordinateur éteint). À noter que lorsqu'on ouvre un fichier en écriture, celui-ci commence par être créé, et s'il existe déjà un fichier du même nom, ce dernier est automatiquement effacé pour être remplacé par le nouveau.

La commande permettant d'écrire dans un fichier est write. Elle s'utilise de la façon suivante :

Écrire dans le fichier	**fichier.write('Bonjour')**
Passer à la ligne puis écrire dans le fichier	**fichier.write('\nBonjour')**
Écrire dans le fichier puis passer à la ligne	**fichier.write('Bonjour\n')**

Remarques :
• À l'intérieur de write doit toujours se trouver une chaîne de caractères.
• Les données écrites sont placées les unes à la suite des autres. Si l'on veut qu'elles soient séparées par des espaces ou des sauts de ligne, il faut donc le préciser.

Par exemple, si on veut créer un fichier test.txt, placé dans le dossier fichiers, contenant 'Bonjour', il faut taper:

```
from os import chdir
chdir("P:/Mes documents/essais_python")
fichier=open("fichiers1/test.txt", 'w')
fichier.write('Bonjour')
fichier.close()
```

IV) LECTURE D'UN FICHIER ET UTILISATION DE SES DONNEES :

Après avoir renommé le fichier test.txt en horla.txt, le déplacer dans le dossier fichiers2 (avec l'explorateur), on ouvre le fichier en lecture par fichier=open("fichiers2/horla.txt", 'r')

Voici ensuite les principales instructions qui permettent de manipuler les données d'un fichier existant :

Faire afficher l'intégralité du fichier	**c=fichier.read() puis print(c)**
Faire afficher n caractères à partir de la position atteinte	**c=fichier.read(n) puis print(c)**
Lire la ligne qui suit à partir de la position atteinte dans le fichier	**fichier.readline()**
Créer la liste des lignes	**fichier.readlines()**
Lire les lignes	**for l in fichier:**

...
Voici ce qu'il faut ajouter au programme précédent pour lire le contenu du fichier horla.txt.

```
fichier=open("fichiers2/horla.txt", 'r')
c=fichier.read()
print(c)
fichier.close()
```

V) LECTURE EN MODE BINAIRE :

En mode texte, un fichier est lu caractère par caractère. En mode binaire, le fichier est lu octet par octet : c'est un peu différent !
Par exemple si on veut la taille d'un fichier, il faut utiliser la lecture en mode binaire'rb' :

```
fichier=open("fichiers2/horla.txt", 'rb')
```

Exécuter l'exemple suivant:

```
fichier=open("fichiers2/horla.txt", 'rb')
data = fichier.read()
print('taille du fichier:', len(data), 'octets')
fichier.close()
```

VI) Sérialisation des données avec le module Pickle :

Le module Pickle est extrêmement pratique pour sauvegarder dans un fichier des structures de données comme les listes (type liste) ou les dictionnaires (type dict).
Un exemple de sauvegarde de dictionnaire :

```
import pickle
departement = {13: 'Marseille', 69: 'Lyon', 75: 'Paris'}
fichier=open("fichiers2/data.txt", 'wb')
pickle.dump(departement,fichier)
fichier.close()
```

L'opération inverse est tout aussi simple :

```
import pickle
fichier=open("fichiers2/data.txt", 'rb')
dept = pickle.load(fichier)
fichier.close()
print(dept)
print(dept[75])
```

VII) METHODES SUPPLEMENTAIRES SUR LES CHAINES DE CARACTERES UTILES LORS DE LA MANIPULATION DE FICHIERS :

• split() : permet de convertir une chaîne chaine1 en une liste de sous-chaînes en choisissant le caractère ou la sous-chaîne chaine2 qui délimite les sous-chaînes. S'utilise par chaine1.split(chaine2) :

```
>>> ch='janvier fevrier mars'
>>> ch.split() # si chaine2 n'est pas précisée, on sépare chaine1 en sous-chaînes selon les espaces
['janvier', 'fevrier', 'mars']
>>> ch.split('e')
['janvi', 'r f', 'vri', 'r mars']
>>> ch.split('er')
['janvi', ' fevri', ' mars']
```

Remarquez que le résultat obtenu est une liste de chaînes de caractères.

• join() : s'utilise par chaine.join(liste), chaine étant une chaîne de caractères et liste une liste dont les éléments sont des chaînes de caractères. Renvoie la chaîne obtenue en insérant chaine entre chaque élément de liste. Le mieux est encore d'observer les exemples :

```
>>> print('._.'.join(['b','cd','fgh','lmnpq']))
b._.cd._.fgh._.lmnpq
>>> print(''.join(['avril','mai','juin','juillet']))
avrilmaijuinjuillet # on a inséré la chaîne vide
>>> print(' '.join(['avril','mai','juin','juillet']))
avril mai juin juillet # on a inséré des espaces
```

• strip() (et ses variantes rstrip() et lstrip()) : permet d'ôter les espaces en début et en fin de chaîne (ou seulement à droite - r pour right - ou à gauche - l pour left -) :

```
>>> ch=" septembre, octobre, novembre "
>>> ch.strip();ch.rstrip();ch.lstrip()
'septembre, octobre, novembre'
' septembre, octobre, novembre'
'septembre, octobre, novembre '
```

De manière plus générale, chaine1.(l ou r)strip(chaine2) permet d'ôter au début et/ou à la fin de chaine1 la chaîne chaine2.

En particulier, pour ôter un marqueur de fin de ligne à la chaîne chaine, on tape : chaine.strip("\n") .

```
>>> ch='AAAOn termine avec decembreAAA' ; print(ch.strip('AAA'))
```

On termine avec decembre

Exercice : ranger dans une liste les nombres présents dans le fichier entiers.txt.
Aide : on définit L=[], puis dans une boucle for, pour chaque ligne,
• on enlève le marqueur de fin de ligne \n à l'aide de la commande strip ;
• on sépare les éléments de la ligne selon la chaîne ", " à l'aide de split ;
• on convertit la liste obtenue (qui est une liste de chaînes) en liste d'entiers ;
• on change L en L à laquelle on a concaténé la liste d'entiers.

VIII) LE MOT CLE WITH :

Avant de passer à un exemple concret, créer un fichier dans un éditeur de texte qui sera enregistré dans le répertoire courant avec le nom zoo.txt et le contenu suivant :
girafe
tigre
singe
souris

Il existe en Python le mot-clé with qui permet d'ouvrir et de fermer un fichier de manière efficace. Si pour une raison ou une autre l'ouverture ou la lecture du fichier conduit à une erreur, l'utilisation de with garantit la bonne fermeture du fichier, ce qui n'est pas le cas dans le code suivant.

```
felins = open("zoo.txt", "r")
>>> lignes = felins.readlines()
>>> lignes
['girafe\n', 'tigre\n', 'singe\n', 'souris\n']
>>> for ligne in lignes:
...         print(ligne)
...
girafe
tigre
singe
souris
>>> felins.close()
```

Voici donc le même exemple avec with :

```
>>> with open("zoo.txt", 'r') as felins:
...         lignes = felins.readlines()
...         for ligne in lignes:
...                 print(ligne)
...
girafe
tigre
singe
souris
```

Remarques :
- L'instruction with introduit un bloc d'indentation. C'est à l'intérieur de ce bloc que nous effectuons toutes les opérations sur le fichier.
- Une fois sorti du bloc d'indentation, Python fermera **automatiquement** le fichier. Vous n'avez donc plus besoin d'utiliser la méthode .close().

A l'avenir, il est fortement conseillé d'utiliser cette méthode pour ouvrir un fichier.

IX) GESTION DES EXCEPTIONS : LES INSTRUCTIONS TRY – EXCEPT – ELSE – FINALLY :

Les exceptions sont les opérations qu'effectue un interpréteur ou un compilateur lorsqu'une erreur est détectée au cours de l'exécution d'un programme. En règle générale, l'exécution du programme est alors interrompue, et un message d'erreur plus ou moins explicite est affiché. Exemple :

```
>>> print(55/0)
ZeroDivisionError: int division or modulo by zero
```

D'autres informations complémentaires sont affichées, indiquant notamment à quel endroit du script l'erreur a été détectée, mais nous ne les reproduisons pas ici.

Le message d'erreur proprement dit comporte deux parties séparées par un double point : d'abord le type d'erreur, et ensuite une information spécifique de cette erreur.
Dans de nombreux cas, il est possible de prévoir à l'avance certaines des erreurs qui risquent de se produire à tel ou tel endroit du programme et d'inclure à cet endroit des instructions particulières, qui seront activées seulement si ces erreurs se produisent. Dans les langages de niveau élevé comme Python, il est également possible d'associer un mécanisme de surveillance à *tout un ensemble d'instructions*, et donc de simplifier le traitement des erreurs qui peuvent se produire dans n'importe laquelle de ces instructions.
Un mécanisme de ce type s'appelle en général *mécanisme de traitement des exceptions*. Celui de Python utilise l'ensemble d'instructions **try - except** – **else**, qui permettent d'intercepter une erreur et d'exécuter une portion de script spécifique de cette erreur. Il fonctionne comme suit.
Le bloc d'instructions qui suit directement une instruction **try** est exécuté par Python *sous réserve*. Si une erreur survient pendant l'exécution de l'une de ces instructions, alors Python annule cette instruction fautive et exécute à sa place le code inclus dans le bloc qui suit l'instruction **except.** Si aucune erreur ne s'est produite dans les instructions qui suivent **try**, alors c'est le bloc qui suit l'instruction **else** qui est exécuté (si cette instruction est présente). **finally** est un bloc qui est exécuté après que tous les autres blocs aient été exécutés, peu importe qu'il y ait eu une exception ou non.
Dans tous les cas, l'exécution du programme peut se poursuivre ensuite avec les instructions ultérieures.

Pour les fichiers on peut suivre les recommandations :

```
try:
   fichier = open('/tmp/fichier', 'w')
except (IOError, OSError):
   # gérer l'erreur
else:
   # faire un truc avec le fichier

# on essaye toujours de fermer notre fichier
finally:
   try:
      fichier.close()

   # le fichier n'a jamais été ouvert et
   # la variable n'existe pas
   except NameError:
      pass
```

Considérons par exemple un script qui demande à l'utilisateur d'entrer un nom de fichier, lequel fichier étant destiné à être ouvert en lecture. Si le fichier n'existe pas, nous ne voulons pas que le programme se « plante ». Nous voulons qu'un avertissement soit affiché, et éventuellement que l'utilisateur puisse essayer d'entrer un autre nom.

```
filename = input("Veuillez entrer un nom de fichier : ")
try:
   f = open(filename, "r")
except:
   print("Le fichier", filename, "est introuvable")
```

Si nous estimons que ce genre de test est susceptible de rendre service à plusieurs endroits d'un programme, nous pouvons aussi l'inclure dans une fonction :

```
def existe(fname):
   try:
      f = open(fname,'r')
      f.close()
      return 1
   except:
      return 0
```

```
filename = input("Veuillez entrer le nom du fichier : ")
if existe(filename):
   print("Ce fichier existe bel et bien.")
else:
   print("Le fichier", filename, "est introuvable.")
```

Il est également possible de faire suivre l'instruction **try** de plusieurs blocs **except**, chacun d'entre eux traitant un type d'erreur spécifique, mais nous ne développerons pas ces compléments ici. Veuillez consulter un ouvrage de référence sur Python si nécessaire.

Remarque :
L'instruction with est utilisée pour encapsuler l'exécution d'un bloc avec des méthodes définies par un gestionnaire de contexte. Cela permet d'utiliser de manière simple le patron de conception classique try….except…finally.
Par exemple :

```
try:
   fichier = open('/tmp/fichier', 'w')
except (IOError, OSError):
   # gérer l'erreur
else:
   # faire un truc avec le fichier
finally:
   try:
      fichier.close()
   except NameError:
      pass
```

Peut être remplacé par :

```
try:
   with open('/tmp/fichier', 'w') as fichier:
      # faire un truc avec le fichier
except EnvironmentError:
   # gérer l'erreur
```

Ce qui est quand même beaucoup plus court. Nous n'allons pas rentrer en détail sur with, mais sachez que beaucoup d'objets qui demandent une fermeture peuvent être traités comme ça.

X) APPLICATIONS :

Exercice 1 :
Écrivez un script qui permette de créer et de relire aisément un fichier texte. Votre programme demandera d'abord à l'utilisateur d'entrer le nom du fichier. Ensuite il lui proposera le choix, soit d'enregistrer de nouvelles lignes de texte, soit d'afficher le contenu du fichier.
L'utilisateur devra pouvoir entrer ses lignes de texte successives en utilisant simplement la touche <Enter> pour les séparer les unes des autres. Pour terminer les entrées, il lui suffira d'entrer une ligne vide (c'est-à-dire utiliser la touche <Enter> seule).
L'affichage du contenu devra montrer les lignes du fichier séparées les unes des autres de la manière la plus naturelle (les codes de fin de ligne ne doivent pas apparaître).

Exercice 2 :
Écrivez un script qui génère automatiquement un fichier texte contenant les tables de multiplication de 2 à 30 (chacune d'entre elles incluant 20 termes seulement).

Exercice 3 :
Écrivez un script qui recopie un fichier texte en triplant tous les espaces entre les mots.

Exercice 4 :
Vous avez à votre disposition un fichier texte dont chaque ligne est la représentation d'une valeur numérique de type réel (mais sans exposants). Par exemple :

14.896
7894.6
123.278
etc.
Écrivez un script qui recopie ces valeurs dans un autre fichier en les arrondissant en nombres entiers (l'arrondi doit être correct).
Faire une version sans with, et une version avec with.

Exercice 5 :
Écrivez un script qui compare les contenus de deux fichiers et signale la première différence rencontrée.
À partir de deux fichiers préexistants A et B, construisez un fichier C qui contienne alternativement un élément de A, un élément de B, un élément de A... et ainsi de suite jusqu'à atteindre la fin de l'un des deux fichiers originaux. Complétez ensuite C avec les éléments restant sur l'autre.

Exercice 6 :
Écrire un script qui combine deux fichiers textes pour en faire un nouveau, tel que la première ligne du fichier nouveau soit la première du premier fichier, puis la deuxième du nouveau soit la première du deuxième, etc…
Utiliser les instructions try, except pour gérer le cas où l'un des fichiers est introuvable.

Exercice 7 :
Écrivez un script qui permette d'encoder un fichier texte dont les lignes contiendront chacune les noms, prénom, adresse, code postal et n° de téléphone de différentes personnes (considérez par exemple qu'il s'agit des membres d'un club).

Exercice 8 :
Écrivez un script qui recopie le fichier utilisé dans l'exercice précédent, en y ajoutant la date de naissance et le sexe des personnes (l'ordinateur devra afficher les lignes une par une et demander à l'utilisateur d'entrer pour chacune les données complémentaires).

Exercice 9 :
Considérons que vous avez fait les exercices précédents et que vous disposez à présent d'un fichier contenant les coordonnées d'un certain nombre de personnes. Écrivez un script qui permette d'extraire de ce fichier les lignes qui correspondent à un code postal bien déterminé.